

Python List Comprehension Quiz

Python List Comprehension Quiz (Questions 1–20)

1. What is the output of `[x for x in range(6) if x % 2 != 0]`?
A) `[1, 3, 5]`
B) `[0, 2, 4]`
C) `[1, 3]`
D) `[0, 1, 2, 3, 4, 5]`
2. Which expression doubles even numbers and keeps odd numbers as-is?
A) `[x * 2 for x in my_list if x % 2 == 0 else x]`
B) `[x * 2 if x % 2 == 0 else x for x in my_list]`
C) `[x * 2 for x in my_list if x % 2 == 0]`
D) `[x if x % 2 != 0 else x * 2 for x in my_list]`
3. How do you create a 3x2 matrix (3 rows, 2 columns) filled with zeros?
A) `[[0 for _ in range(3)] for _ in range(2)]`
B) `[0 for _ in range(2) for _ in range(3)]`
C) `[[0 for _ in range(2)] for _ in range(3)]`
D) `[[0] * 3 for _ in range(2)]`
4. Given matrix = `[[1, 2], [3, 4]]`, which flattens it into `[1, 2, 3, 4]`?
A) `[val for val in row for row in matrix]`
B) `[val for row in matrix for val in row]`
C) `[[val for val in row] for row in matrix]`
D) `[val for row in matrix if val in row]`
5. True or False: A list comprehension modifies the original list in-place.
A) True
B) False
6. How do you filter for numbers that are both greater than 10 AND even?
A) `[x for x in nums if x > 10 if x % 2 == 0]`
B) `[x for x in nums if x > 10 or x % 2 == 0]`
C) `[x for x in nums while x > 10 and x % 2 == 0]`
D) `[if x > 10 and x % 2 == 0 then x for x in nums]`
7. Which structure correctly adds 1 to every integer in a 3D list cube?
A) `[v + 1 for block in cube for row in block for v in row]`
B) `[[[v + 1 for v in row] for row in block] for block in cube]`
C) `[[v + 1 for v in row for row in block for block in cube]]`
D) `[[[v + 1] for v in row] for row in block for block in cube]`
8. If matrix = `[[10, 20], [30, 40]]`, how do you access the value 30?
A) `matrix[0][1]`
B) `matrix[1][0]`
C) `matrix[1, 0]`
D) `matrix{1}{0}`
9. What is the result of `[i*j for i in [1, 2] for j in [10, 20]]`?
A) `[10, 20, 20, 40]`
B) `[10, 40]`
C) `[[10, 20], [20, 40]]`
D) `[11, 21, 12, 22]`

10. True or False: Use a list comprehension to update a single value at `matrix[0][0]`.
A) True
B) False
11. Which correctly transposes a 2x3 matrix into a 3x2 matrix?
A) `[[row[i] for row in matrix] for i in range(len(matrix[0]))]`
B) `[[matrix[i][row] for i in range(3)] for row in matrix]`
C) `[row[i] for i in range(3) for row in matrix]`
D) `[[row for row in matrix] for i in range(3)]`
12. Given `data = [[[1], [2]], [[3], [4]]]`, what index path reaches the value 3?
A) `data[0][1][0]`
B) `data[1][0][0]`
C) `data[1][1][0]`
D) `data[1][0][1]`
13. Which expression converts `['apple', 'be', 'cherry']` to `['APPLE', 'be', 'CHERRY']`?
A) `[s.upper() for s in words if len(s) > 2]`
B) `[s.upper() if len(s) > 2 else s for s in words]`
C) `[if len(s) > 2 s.upper() else s for s in words]`
D) `[s.upper() for s in words]`
14. How do you keep ONLY positive numbers within nested sublists `[[1, -1], [2, -2]]`?
A) `[x for g in groups for x in g if x > 0]`
B) `[[x for x in g if x > 0] for g in groups]`
C) `[[x if x > 0 for x in g] for g in groups]`
D) `[g for g in groups if all(x > 0 for x in g)]`
15. What is the risk of using `grid = [[0]*3]*3` to initialize a 2D list?
A) It creates a tuple instead of a list.
B) Updating `grid[0][0]` will update the first element of every row due to shared references.
C) It is significantly slower than a list comprehension.
D) It only creates a 1D list of 9 zeros.
16. To save memory with a large range (e.g., 10 million), what should you use instead of `[]`?
A) `list(range(10000000))`
B) `(x for x in range(10000000))`
C) `{x for x in range(10000000)}`
D) `[[x] for x in range(10000000)]`
17. What is the output of `[name[:3] for name in ['Python', 'Ruby']]`?
A) `['Pyt', 'Rub']`
B) `['on', 'by']`
C) `['Pyth', 'Ruby']`
D) `['P', 'y', 't', 'R', 'u', 'b']`
18. Which result is produced by `[[i for i in range(j)] for j in range(3)]`?
A) `[[0], [0, 1], [0, 1, 2]]`
B) `[[], [0], [0, 1]]`
C) `[[0, 1, 2], [0, 1, 2], [0, 1, 2]]`
D) `[0, 1, 2]`
19. How do you find the intersection of two lists `list1` and `list2`?
A) `[x for x in list1 if x in list2]`
B) `[x if x in list2 for x in list1]`
C) `[list1 + list2]`

D) [x for x in list1 or list2]

20. Why are list comprehensions often preferred over standard for loops?

- A) They are always faster because they bypass the Python interpreter.
- B) They are more readable for simple transformations and optimized internally.
- C) They automatically handle memory management for large datasets.
- D) They can perform operations that for loops cannot.